Auto Email / Ticket Categorizer

GITHUB REPOSITORY
Padmanaban29072004/ticket_categorizer
https://github.com/Padmanaban29072004/ticket_categorizer

Lightweight NLP classifier that reads an incoming support ticket (subject + body) and routes it to Billing, Technical, HR, or General — with confidence scores, a human-review threshold, and simple priority tagging.

STACK	MODEL	HOLDOUT ACC.	DEMO HITS
Python · sklearn	**TF-IDF + LogReg**	**62.5%**	**5 / 5 labels**

1. Problem & goal

Enterprise helpdesks receive a mixed stream of tickets every day. Manual triage is slow and inconsistent. This project builds a lightweight routing layer that reads new ticket text and assigns a department automatically — the same pattern used in front of live ticket queues.

- Input: ticket subject + body text
- Output: category (Billing / Technical / HR / General)
- Also return confidence %, human-review flag, and urgent/normal priority
- Must classify one ticket on demand (not only a static test set)

2. What we built (how)

End-to-end pipeline: clean text → TF-IDF features → Logistic Regression → evaluate on a stratified holdout → retrain on all labeled data → serve predictions via API helper and an interactive CLI.

Stage	What happens
1. Data	32 labeled tickets in data/tickets.csv (8 per class: Billing, Technical, HR, General)
2. Clean	Lowercase; strip HTML, URLs, emails, punctuation, stopwords (src/preprocess.py)
3. Features	Combine subject+body; TF-IDF unigrams + bigrams
4. Model	Logistic Regression (C=8, max_iter=2000) — sharper probs than Naive Bayes here
5. Evaluate	25% stratified holdout → accuracy, precision/recall/F1, confusion matrix
6. Ship	Retrain on 100% of labels; save models/vectorizer.joblib + model.joblib

Stage	What happens
7. Infer	predict_ticket() cleans text the same way as training, then scores one ticket
8. Bonuses	Confidence %; review if < 60%; keyword priority; live CLI (--interactive)

DESIGN CHOICE

Training and inference share the same clean_text() path so the vectorizer sees the same token distribution. After reporting holdout metrics, the final saved model is fit on all labeled rows so production inference uses every example.

3. Evaluation outcomes

Holdout set: 8 tickets (2 per class), stratified, random_state=42. With only 32 total samples, holdout numbers are noisy — they are a sanity check, not a production SLA. The live demo on clear tickets is the stronger usability signal.

ACCURACY	MACRO F1	BILLING	TECHNICAL
62.5%	**~0.64**	**P1.00 R1.00**	**P1.00 R0.50**

```
Confusion matrix  (rows = true, cols = pred)
Labels: [Billing, General, HR, Technical]

 [[2 0 0 0]     Billing: both correct
  [0 1 1 0]     General: 1 correct, 1 → HR
  [0 1 1 0]     HR:      1 correct, 1 → General
  [0 0 1 1]]    Technical: 1 correct, 1 → HR
```

INSIGHT FROM THE MATRIX

Billing is cleanly separable (invoice/refund/payment language). Most errors are General ↔ HR swaps — short, polite questions that share vocabulary ("how", "policy", "account"). Technical recall dipped when wording overlapped HR/General. More real tickets in those grey zones would help most.

4. Live prediction outcomes

Five unseen sample tickets (assessment requirement). Labels match intent; confidence drives auto-assign vs human review.

```
SUBJECT: Invoice missing
 -> Billing   conf=0.70  review=False  priority=urgent   ("asap")
SUBJECT: App crashes on save
 -> Technical conf=0.73  review=False  priority=normal
SUBJECT: Leave balance query
 -> HR        conf=0.67  review=False  priority=normal
SUBJECT: Refund status
 -> Billing   conf=0.52  review=True   priority=normal   (borderline)
SUBJECT: Password reset
 -> Technical conf=0.73  review=False  priority=normal
```

Outcome	Meaning
4 / 5 auto-assign	Clear tickets cleared the ~60% confidence bar and route automatically
1 human review	"Refund status" stayed Billing but flagged for a person (conf 52%)
Priority layer	"asap" / "down" / "urgent" / "outage" mark urgent without changing category
CLI demo	python src/predict.py --interactive types a ticket and routes instantly

5. Key insights

- TF-IDF + linear models are enough for short support text — fast to train, easy to explain.
- Logistic Regression beat Multinomial Naive Bayes here on usable confidence scores (NB stayed ~0.4–0.5 even on clear tickets, so almost everything hit human review).
- A 60% threshold is conservative on tiny multiclass data; that is desirable for a triage tool — wrong auto-routes are costlier than an extra human look.
- Edge cases (gibberish / vague text) get low confidence → needs_human_review=True instead of forcing a department.
- Keyword priority is a cheap second signal; it should eventually be learned from labeled urgency.

6. Quick start

From the project root:

```
python -m venv venv
# Windows
venv\Scripts\python.exe -m pip install -r requirements.txt
venv\Scripts\python.exe src\train.py
venv\Scripts\python.exe notebooks\ticket_categorizer_demo.py
venv\Scripts\python.exe src\predict.py --interactive

# macOS / Linux
venv/bin/python -m pip install -r requirements.txt
venv/bin/python src/train.py
venv/bin/python notebooks/ticket_categorizer_demo.py
venv/bin/python src/predict.py --interactive
```

7. Project layout

```
├── data/tickets.csv                # 32 labeled tickets
├── src/preprocess.py               # text cleaning
├── src/train.py                    # train + evaluate + save
├── src/predict.py                  # API + interactive CLI
├── notebooks/ticket_categorizer_demo.py
├── models/                         # gitignored artifacts
├── requirements.txt
└── README.md
```

8. Approach summary

Cleaned subject+body text, vectorized with TF-IDF (1–2 grams), and trained Logistic Regression for fast short-text classification with usable probability estimates. Predictions return the category plus a confidence score; below ~60% confidence the ticket is flagged for human review rather than auto-routed. Priority (urgent / normal) is a lightweight keyword layer (urgent, down, asap, not working, critical, outage).

9. Reflection — what next

With more real helpdesk data I would expand coverage of ambiguous General vs Technical tickets and rare failure phrases. Next I'd calibrate probabilities, compare linear SVM / Multinomial Naive Bayes on a larger set, and learn priority instead of hard-coded keywords. A human-in-the-loop queue for low-confidence tickets, plus periodic retraining on reviewed labels, would make routing more reliable in production.

10. Requirements

- Python 3.9+
- scikit-learn, pandas, joblib (see requirements.txt)

Fobes Skill Itech Pvt Ltd — AI/ML Internship Program · Technical Assessment
Repository: https://github.com/Padmanaban29072004/ticket_categorizer
Document generated from project results + README.